

Sistema de Gestión Hotelera

Documento Técnico — Primera Entrega

Documento de Análisis, Modelo Relacional y Estructura Base en Spring Boot

Luis Miguel Viuche	20212020082
María Fernanda Vélez Benítez	20232020104
Sharon Stefany Castilla Araque	20251020089
David Santiago Ospina Carrillo	20232020013

Curso: Programación Avanzada con Java y Spring Boot

Grupo ACM

2026

Índice

1. Introducción	2
2. Objetivo General	2
3. Objetivos Específicos	2
4. Alcance del Sistema	2
5. Levantamiento de Requerimientos Funcionales	2
5.1. Gestión de Usuarios	2
5.2. Gestión de Hoteles	3
5.3. Gestión de Habitaciones	3
5.4. Gestión de Reservas	4
5.5. Gestión de Pagos	4
5.6. Facturación	4
5.7. Gestión de Servicios Adicionales	5
6. Requerimientos No Funcionales	5
7. Identificación de Entidades Principales	6
8. Modelo Relacional	6
8.1. Diagrama Relacional	6
8.2. Relaciones entre Entidades	7
8.3. Claves Primarias y Foráneas por Tabla	8
8.4. Justificación de Tablas Intermedias	9
9. Estructura del Proyecto	9
10. Dependencias del Proyecto	10
11. Configuración de Persistencia	11
12. Migraciones Automáticas con Flyway	11
12.1. V1__crear_tablas.sql	11
12.2. V2__restricciones_e_indices.sql	13
12.3. V3__datos_iniciales.sql	14
13. Entidades JPA	14
13.1. CadenaHotel	15
13.2. Sucursal	15
13.3. Rol y Usuario	15
13.4. TipoHabitacion y Habitacion	16
13.5. Reserva y tablas intermedias	17
13.6. Pago y Factura	18
14. Conclusiones	19

1. Introducción

El Sistema de Gestión Hotelera es una solución de software backend desarrollada con **Spring Boot** y **PostgreSQL**, orientada a la administración integral de establecimientos hoteleros. El sistema centraliza los procesos operativos clave: gestión de usuarios, habitaciones, reservas, pagos, facturación y servicios adicionales, con una arquitectura diseñada para escalar a múltiples cadenas hoteleras y sus respectivas sucursales.

Esta primera entrega cubre la estructura base del sistema: el modelo relacional de base de datos, las entidades JPA, la configuración de persistencia y las migraciones automáticas mediante Flyway.

2. Objetivo General

Desarrollar la estructura base de un sistema hotelero utilizando Spring Boot y PostgreSQL, implementando un modelo relacional robusto, escalable y alineado con las necesidades reales de la administración hotelera.

3. Objetivos Específicos

- Diseñar una base de datos relacional normalizada hasta al menos la Tercera Forma Normal (3FN).
- Identificar y modelar las entidades principales del dominio hotelero.
- Definir relaciones entre tablas con integridad referencial.
- Implementar las entidades utilizando JPA/Hibernate.
- Configurar migraciones automáticas de esquema de base de datos con Flyway.
- Preparar la arquitectura inicial del backend para futuras funcionalidades (API REST, seguridad, reportes).

4. Alcance del Sistema

La Tabla 1 describe los módulos funcionales cubiertos en esta primera entrega.

5. Levantamiento de Requerimientos Funcionales

5.1. Gestión de Usuarios

Cuadro 1: Módulos del sistema

Módulo	Descripción
Usuarios	Registro y autenticación de clientes, empleados y administradores
Hoteles	Administración de cadenas hoteleras y sucursales
Habitaciones	Gestión de tipos, precios, disponibilidad y estado
Reservas	Ciclo completo: crear, modificar, cancelar, finalizar
Pagos	Registro de pagos con múltiples métodos
Facturación	Generación y consulta de facturas asociadas a pagos
Servicios	Registro y asociación de servicios adicionales a reservas

Cuadro 2: Requerimientos funcionales – Usuarios

ID	Requerimiento
RF-01	El sistema debe permitir el registro de clientes con sus datos personales
RF-02	El sistema debe permitir el registro de empleados asociados a una sucursal
RF-03	El sistema debe gestionar administradores del sistema
RF-04	El sistema debe permitir el inicio de sesión con credenciales
RF-05	El sistema debe controlar roles y permisos: cliente, empleado, administrador

5.2. Gestión de Hoteles

Cuadro 3: Requerimientos funcionales – Hoteles

ID	Requerimiento
RF-06	El sistema debe permitir registrar cadenas hoteleras con nombre, descripción y estado
RF-07	El sistema debe permitir administrar sucursales de una misma cadena, con nombre, dirección, ciudad, teléfono, correo, categoría y estado propios

5.3. Gestión de Habitaciones

Cuadro 4: Requerimientos funcionales – Habitaciones

ID	Requerimiento
RF-08	El sistema debe permitir registrar habitaciones con número, piso, capacidad, precio y estado
RF-09	Las habitaciones deben clasificarse por tipo: sencilla, doble, suite, presidencial
RF-10	El sistema debe gestionar la disponibilidad de cada habitación

5.4. Gestión de Reservas

Cuadro 5: Requerimientos funcionales – Reservas

ID	Requerimiento
RF-11	El sistema debe permitir crear reservas asociando un cliente a una o varias habitaciones
RF-12	El sistema debe permitir consultar disponibilidad por fechas
RF-13	El sistema debe permitir modificar fechas y datos de una reserva existente
RF-14	El sistema debe permitir cancelar reservas
RF-15	Las reservas deben manejar estados: pendiente, confirmada, cancelada, finalizada

5.5. Gestión de Pagos

Cuadro 6: Requerimientos funcionales – Pagos

ID	Requerimiento
RF-16	El sistema debe registrar pagos asociados a una reserva (relación 1:1)
RF-17	Los pagos deben soportar métodos: efectivo, tarjeta, transferencia, pago digital
RF-18	El sistema debe registrar el monto, fecha y estado del pago

5.6. Facturación

Cuadro 7: Requerimientos funcionales – Facturación

ID	Requerimiento
RF-19	El sistema debe generar facturas vinculadas a un pago (relación 1:1)
RF-20	Cada factura debe incluir fecha de emisión, subtotal, impuestos, total y descripción
RF-21	El sistema debe permitir consultar el historial de facturación

5.7. Gestión de Servicios Adicionales

Cuadro 8: Requerimientos funcionales – Servicios

ID	Requerimiento
RF-22	El sistema debe registrar servicios adicionales: restaurante, spa, lavandería, transporte
RF-23	Los servicios deben asociarse a reservas específicas con cantidad y precio mediante la tabla intermedia <code>reserva_servicio</code>

6. Requerimientos No Funcionales

Cuadro 9: Requerimientos no funcionales

ID	Requerimiento	Categoría
RNF-01	El sistema debe ser escalable para múltiples cadenas y sucursales	Escalabilidad
RNF-02	La base de datos debe estar normalizada (mínimo 3FN)	Integridad
RNF-03	El backend debe desarrollarse con Spring Boot	Tecnología
RNF-04	La persistencia debe manejarse con JPA/Hibernate	Tecnología
RNF-05	La base de datos será PostgreSQL	Tecnología
RNF-06	El sistema debe implementar migraciones automáticas de esquema	Mantenibilidad
RNF-07	La arquitectura debe facilitar mantenimiento y expansión futura	Mantenibilidad
RNF-08	El código debe estar alojado en GitHub, organizado y documentado	Colaboración

7. Identificación de Entidades Principales

Cuadro 10: Entidades del dominio hotelero

Entidad	Descripción
CadenaHotel	Cadena o marca hotelera registrada en el sistema
Sucursal	Sede física de una cadena, con datos propios de contacto y ubicación
Rol	Define el nivel de acceso de un usuario
Usuario	Persona que interactúa con el sistema (cliente, empleado, admin)
TipoHabitacion	Clasificación de habitación: sencilla, doble, suite, presidencial
Habitacion	Unidad de alojamiento perteneciente a una sucursal
Reserva	Registro del alojamiento de un cliente en una o varias habitaciones
ReservaHabitacion	Tabla intermedia: asocia habitaciones a una reserva (1:N)
Servicio	Servicio adicional ofrecido por el hotel
ReservaServicio	Tabla intermedia: asocia servicios a reservas con cantidad y precio
Pago	Transacción económica asociada a una reserva (1:1)
Factura	Documento fiscal generado a partir de un pago (1:1)

8. Modelo Relacional

8.1. Diagrama Relacional

La Figura 1 muestra el diagrama relacional completo del sistema, con todas las entidades, atributos y relaciones definidas para la base de datos del Sistema de Gestión Hotelera.

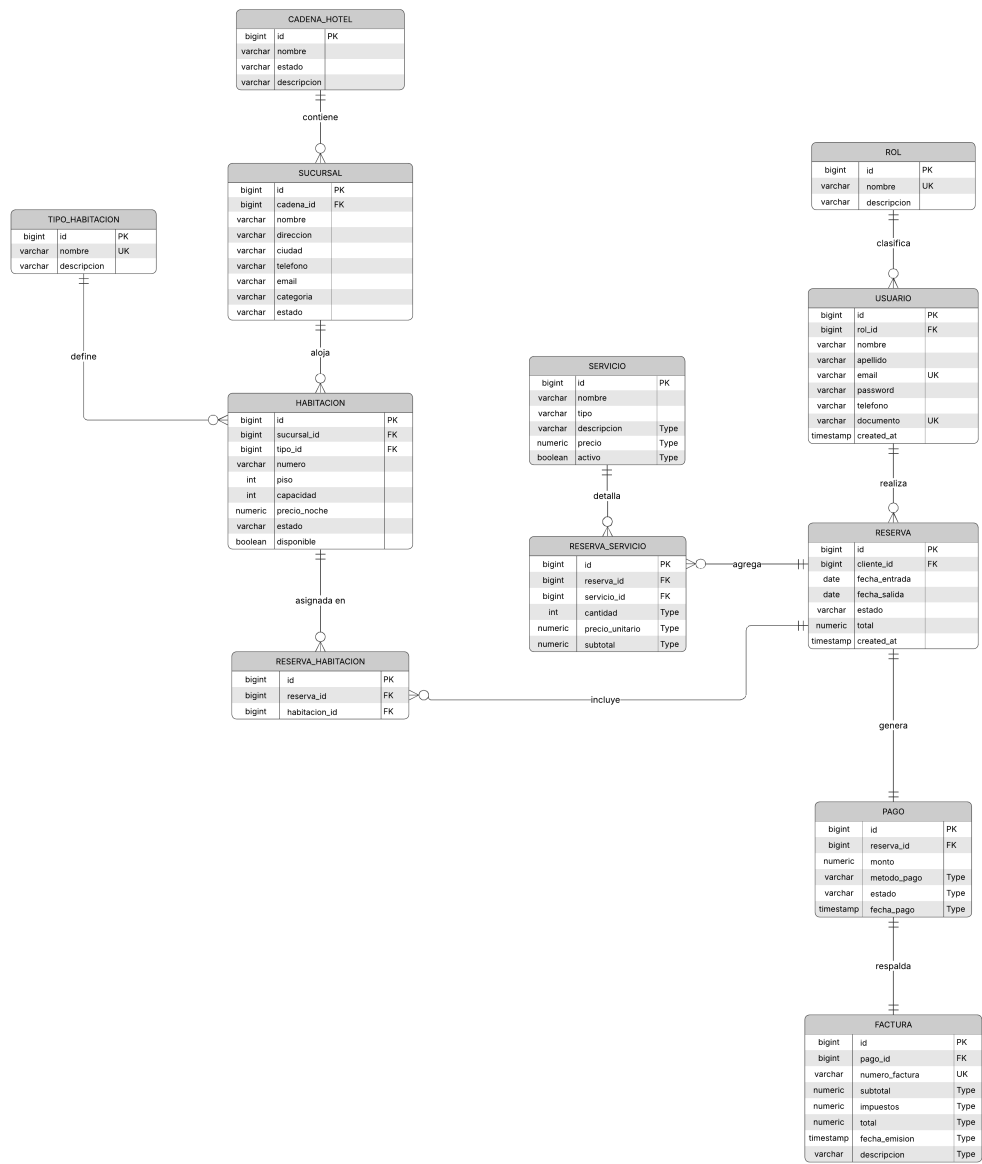


Figura 1: Diagrama relacional completo del Sistema de Gestión Hotelera

8.2. Relaciones entre Entidades

Cuadro 11: Relaciones del modelo relacional

Tabla origen	Relación	Tabla destino	Campo de unión
CadenaHotel	1 a muchos	Sucursal	sucursal.cadena_id → cadena_hotel.id
Sucursal	1 a muchos	Habitacion	habitacion.sucursal_id → sucursal.id
TipoHabitacion	1 a muchos	Habitacion	habitacion.tipo_id → tipo_habitacion.id
Rol	1 a muchos	Usuario	usuario.rol_id → rol.id
Usuario	1 a muchos	Reserva	reserva.cliente_id → usuario.id
Reserva	1 a muchos	ReservaHabitacion	reserva_habitacion.reserva_id → reserva.id
Habitacion	1 a muchos	ReservaHabitacion	reserva_habitacion.habitacion_id → habitacion.id
Reserva	1 a muchos	ReservaServicio	reserva_servicio.reserva_id → reserva.id
Servicio	1 a muchos	ReservaServicio	reserva_servicio.servicio_id → servicio.id
Reserva	1 a 1	Pago	pago.reserva_id → reserva.id
Pago	1 a 1	Factura	factura.pago_id → pago.id

8.3. Claves Primarias y Foráneas por Tabla

Todas las tablas usan `id` `bigserial` como **clave primaria (PK)**. Las **claves foráneas (FK)** son los campos que apuntan al `id` de otra tabla, creando las relaciones del modelo.

Cuadro 12: PKs y FKs del modelo relacional

Tabla	Campo	Tipo llave	Referencia
cadena_hotel	id	PK	—
sucursal	id	PK	—
	cadena_id	FK	cadena_hotel.id
rol	id	PK	—
usuario	id	PK	—
	rol_id	FK	rol.id
tipo_habitacion	id	PK	—
habitacion	id	PK	—
	sucursal_id	FK	sucursal.id
	tipo_id	FK	tipo_habitacion.id

Tabla	Campo	Tipo llave	Referencia
reserva	id	PK	—
	cliente_id	FK	usuario.id
reserva_habitacion	id	PK	—
	reserva_id	FK	reserva.id
	habitacion_id	FK	habitacion.id
servicio	id	PK	—
reserva_servicio	id	PK	—
	reserva_id	FK	reserva.id
	servicio_id	FK	servicio.id
pago	id	PK	—
	reserva_id	FK (único)	reserva.id
factura	id	PK	—
	pago_id	FK (único)	pago.id

*Nota: Los campos FK marcados como “único” implementan una restricción **UNIQUE** en la base de datos, garantizando las relaciones 1 a 1 entre Reserva–Pago y Pago–Factura.*

8.4. Justificación de Tablas Intermedias

ReservaHabitacion: Una reserva puede incluir varias habitaciones, y una habitación es una unidad única que no puede estar ocupada por dos reservas al mismo tiempo. Para representar esta relación sin violar la integridad, se usa una tabla intermedia que vincula **reserva** con **habitacion**.

ReservaServicio: Una reserva puede incluir varios servicios adicionales (spa, restaurante, etc.) y un servicio puede ser solicitado en muchas reservas. Esta tabla intermedia resuelve la relación muchos a muchos y agrega atributos propios de cada asociación: **cantidad**, **precio_unitario** y **subtotal**.

9. Estructura del Proyecto

```

1 ProyectoHotel/
2 |-- src/
3 |   |-- main/
4 |       |-- java/com/acm/proyectohotel/
5 |           |-- entidad/                # Entidades JPA (12 clases)
6 |           |-- enums/                  # Enumeraciones de dominio
7 |           |-- ProyectoHotelApplication.java
8 |           |-- resources/
9 |               |-- db/migration/        # Scripts Flyway (V1, V2, V3)
10 |               |-- application.properties
11 |               |-- application-test.properties
12 |           |-- test/
13 |-- docs/                               # Documentacion del proyecto
14 |-- pom.xml

```

```
15 ' -- README.md
```

Listing 1: Estructura de directorios del proyecto

10. Dependencias del Proyecto

Las siguientes dependencias conforman el pom.xml del proyecto, bajo Spring Boot 4.0.6 y Java 17:

```
1 <!-- Spring Data JPA + Hibernate -->
2 <dependency>
3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter-data-jpa</artifactId>
5 </dependency>
6
7 <!-- Spring Web MVC -->
8 <dependency>
9     <groupId>org.springframework.boot</groupId>
10    <artifactId>spring-boot-starter-webmvc</artifactId>
11 </dependency>
12
13 <!-- Flyway (migraciones automaticas) -->
14 <dependency>
15     <groupId>org.flywaydb</groupId>
16     <artifactId>flyway-core</artifactId>
17 </dependency>
18 <dependency>
19     <groupId>org.flywaydb</groupId>
20     <artifactId>flyway-database-postgresql</artifactId>
21 </dependency>
22
23 <!-- Driver PostgreSQL -->
24 <dependency>
25     <groupId>org.postgresql</groupId>
26     <artifactId>postgresql</artifactId>
27     <scope>runtime</scope>
28 </dependency>
29
30 <!-- Lombok -->
31 <dependency>
32     <groupId>org.projectlombok</groupId>
33     <artifactId>lombok</artifactId>
34     <scope>provided</scope>
35 </dependency>
36
37 <!-- H2 para pruebas -->
38 <dependency>
39     <groupId>com.h2database</groupId>
40     <artifactId>h2</artifactId>
41     <scope>test</scope>
42 </dependency>
```

Listing 2: Dependencias en pom.xml

11. Configuración de Persistencia

```
1 spring.application.name=ProyectoHotel
2
3 spring.datasource.url=${URL_BD:jdbc:postgresql://localhost:5432/
  proyecto_hotel}
4 spring.datasource.username=${USUARIO_BD:postgres}
5 spring.datasource.password=${CLAVE_BD:postgres}
6
7 # Flyway es el dueño del esquema; Hibernate solo valida.
8 spring.jpa.hibernate.ddl-auto=validate
9 spring.jpa.open-in-view=false
10 spring.jpa.show-sql=true
11
12 spring.flyway.enabled=true
13 spring.flyway.locations=classpath:db/migration
```

Listing 3: application.properties

12. Migraciones Automáticas con Flyway

Las migraciones se ejecutan automáticamente al iniciar la aplicación en el siguiente orden:

12.1. V1__crear_tablas.sql

```
1 create table cadena_hotel (
2     id bigserial primary key,
3     nombre varchar(100) not null,
4     descripcion varchar(255),
5     estado varchar(30) not null
6 );
7
8 create table sucursal (
9     id bigserial primary key,
10    cadena_id bigint not null,
11    nombre varchar(100) not null,
12    direccion varchar(150) not null,
13    ciudad varchar(80) not null,
14    telefono varchar(30) not null,
15    email varchar(120) not null,
16    categoria varchar(30) not null,
17    estado varchar(30) not null,
18    constraint fk_sucursal_cadena
19        foreign key (cadena_id) references cadena_hotel (id)
20 );
21
22 create table rol (
23     id bigserial primary key,
24     nombre varchar(50) not null,
25     descripcion varchar(255)
26 );
27
```

```
28 create table usuario (  
29     id bigserial primary key,  
30     rol_id bigint not null,  
31     nombre varchar(80) not null,  
32     apellido varchar(80) not null,  
33     email varchar(120) not null,  
34     password varchar(120) not null,  
35     telefono varchar(30) not null,  
36     documento varchar(30) not null,  
37     created_at timestamp not null default now(),  
38     constraint fk_usuario_rol  
39         foreign key (rol_id) references rol (id)  
40 );  
41  
42 create table tipo_habitacion (  
43     id bigserial primary key,  
44     nombre varchar(50) not null,  
45     descripcion varchar(255)  
46 );  
47  
48 create table habitacion (  
49     id bigserial primary key,  
50     sucursal_id bigint not null,  
51     tipo_id bigint not null,  
52     numero varchar(20) not null,  
53     piso int not null,  
54     capacidad int not null,  
55     precio_noche numeric(12, 2) not null,  
56     estado varchar(30) not null,  
57     disponible boolean not null,  
58     constraint fk_habitacion_sucursal  
59         foreign key (sucursal_id) references sucursal (id),  
60     constraint fk_habitacion_tipo  
61         foreign key (tipo_id) references tipo_habitacion (id)  
62 );  
63  
64 create table reserva (  
65     id bigserial primary key,  
66     cliente_id bigint not null,  
67     fecha_entrada date not null,  
68     fecha_salida date not null,  
69     estado varchar(30) not null,  
70     total numeric(12, 2) not null,  
71     created_at timestamp not null default now(),  
72     constraint fk_reserva_cliente  
73         foreign key (cliente_id) references usuario (id)  
74 );  
75  
76 create table reserva_habitacion (  
77     id bigserial primary key,  
78     reserva_id bigint not null,  
79     habitacion_id bigint not null,  
80     constraint fk_reserva_habitacion_reserva  
81         foreign key (reserva_id) references reserva (id),  
82     constraint fk_reserva_habitacion_habitacion  
83         foreign key (habitacion_id) references habitacion (id)  
84 );  
85
```

```

86 create table servicio (
87     id bigserial primary key,
88     nombre varchar(80) not null,
89     tipo varchar(50) not null,
90     descripcion varchar(255),
91     precio numeric(12, 2) not null,
92     activo boolean not null
93 );
94
95 create table reserva_servicio (
96     id bigserial primary key,
97     reserva_id bigint not null,
98     servicio_id bigint not null,
99     cantidad int not null,
100     precio_unitario numeric(12, 2) not null,
101     subtotal numeric(12, 2) not null,
102     constraint fk_reserva_servicio_reserva
103         foreign key (reserva_id) references reserva (id),
104     constraint fk_reserva_servicio_servicio
105         foreign key (servicio_id) references servicio (id)
106 );
107
108 create table pago (
109     id bigserial primary key,
110     reserva_id bigint not null,
111     monto numeric(12, 2) not null,
112     metodo_pago varchar(30) not null,
113     estado varchar(30) not null,
114     fecha_pago timestamp not null,
115     constraint fk_pago_reserva
116         foreign key (reserva_id) references reserva (id),
117     constraint uq_pago_reserva unique (reserva_id)
118 );
119
120 create table factura (
121     id bigserial primary key,
122     pago_id bigint not null,
123     numero_factura varchar(50) not null,
124     subtotal numeric(12, 2) not null,
125     impuestos numeric(12, 2) not null,
126     total numeric(12, 2) not null,
127     fecha_emision timestamp not null,
128     descripcion varchar(255),
129     constraint fk_factura_pago
130         foreign key (pago_id) references pago (id),
131     constraint uq_factura_pago unique (pago_id)
132 );

```

Listing 4: Creación de tablas principales

12.2. V2__restricciones_e_indices.sql

```

1 -- Unicidad
2 alter table usuario add constraint uq_usuario_email unique (email);
3 alter table usuario add constraint uq_usuario_documento unique (
    documento);

```

```

4 alter table rol      add constraint uq_rol_nombre unique (nombre);
5 alter table factura add constraint uq_factura_numero unique (
    numero_factura);
6
7 -- Dominios controlados (enums)
8 alter table cadena_hotel add constraint chk_cadena_estado
9     check (estado in ('ACTIVO', 'INACTIVO'));
10 alter table habitacion add constraint chk_habitacion_estado
11     check (estado in ('DISPONIBLE', 'OCUPADA', 'MANTENIMIENTO', 'LIMPIEZA
    '));
12 alter table reserva add constraint chk_reserva_estado
13     check (estado in ('PENDIENTE', 'CONFIRMADA', 'CANCELADA', 'FINALIZADA
    '));
14 alter table pago add constraint chk_pago_metodo
15     check (metodo_pago in ('EFECTIVO', 'TARJETA', 'TRANSFERENCIA', '
    PAGO_DIGITAL'));
16 alter table servicio add constraint chk_servicio_tipo
17     check (tipo in ('RESTAURANTE', 'SPA', 'LAVANDERIA', 'TRANSPORTE'));
18
19 -- Reglas de negocio
20 alter table reserva add constraint chk_reserva_fechas
21     check (fecha_salida > fecha_entrada);
22 alter table pago      add constraint chk_pago_monto check (monto > 0);
23
24 -- Indices de clave foranea
25 create index idx_sucursal_cadena      on sucursal (cadena_id);
26 create index idx_usuario_rol          on usuario (rol_id);
27 create index idx_habitacion_sucursal on habitacion (sucursal_id);
28 create index idx_reserva_cliente      on reserva (cliente_id);

```

Listing 5: Restricciones de integridad e índices

12.3. V3__datos_iniciales.sql

```

1 insert into rol (nombre, descripcion) values
2     ('CLIENTE',      'Usuario que realiza y gestiona sus reservas'),
3     ('EMPLEADO',      'Personal operativo de una sucursal'),
4     ('ADMINISTRADOR', 'Gestiona hoteles, usuarios y el sistema');
5
6 insert into tipo_habitacion (nombre, descripcion) values
7     ('SENCILLA',      'Habitacion individual basica'),
8     ('DOBLE',          'Habitacion para dos personas'),
9     ('SUITE',          'Suite con sala independiente'),
10    ('PRESIDENCIAL',    'Suite presidencial de lujo');

```

Listing 6: Datos de catálogo iniciales

13. Entidades JPA

Cada entidad está anotada con `@Entity`, `@Table`, y usa Lombok (`@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`) para reducir código repetitivo.

13.1. CadenaHotel

```
1 @Entity
2 @Table(name = "cadena_hotel")
3 @Data @NoArgsConstructor @AllArgsConstructor
4 public class CadenaHotel {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false)
10    private String nombre;
11
12    private String descripcion;
13
14    @Enumerated(EnumType.STRING)
15    @Column(nullable = false, length = 30)
16    private EstadoGeneral estado;
17 }
```

Listing 7: Entidad CadenaHotel.java

13.2. Sucursal

```
1 @Entity
2 @Table(name = "sucursal")
3 @Data @NoArgsConstructor @AllArgsConstructor
4 public class Sucursal {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @ManyToOne(fetch = FetchType.LAZY)
10    @JoinColumn(name = "cadena_id", nullable = false)
11    private CadenaHotel cadenaHotel;
12
13    @Column(nullable = false) private String nombre;
14    @Column(nullable = false) private String direccion;
15    @Column(nullable = false) private String ciudad;
16    @Column(nullable = false) private String telefono;
17    @Column(nullable = false) private String email;
18    @Column(nullable = false) private String categoria;
19
20    @Enumerated(EnumType.STRING)
21    @Column(nullable = false, length = 30)
22    private EstadoGeneral estado;
23 }
```

Listing 8: Entidad Sucursal.java

13.3. Rol y Usuario

```
1 @Entity @Table(name = "rol")
2 @Data @NoArgsConstructor @AllArgsConstructor
```



```
3 public class Rol {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
6     @Column(nullable = false) private String nombre;
7     private String descripcion;
8 }
```

Listing 9: Entidad Rol.java

```
1 @Entity @Table(name = "usuario")
2 @Data @NoArgsConstructor @AllArgsConstructor
3 public class Usuario {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
6
7     @ManyToOne(fetch = FetchType.LAZY)
8     @JoinColumn(name = "rol_id", nullable = false)
9     private Rol rol;
10
11     @Column(nullable = false) private String nombre;
12     @Column(nullable = false) private String apellido;
13     @Column(nullable = false) private String email;
14     @Column(nullable = false) private String password;
15     @Column(nullable = false) private String telefono;
16     @Column(nullable = false) private String documento;
17
18     @CreationTimestamp
19     @Column(name = "created_at", nullable = false, updatable = false)
20     private LocalDateTime createdAt;
21 }
```

Listing 10: Entidad Usuario.java

13.4. TipoHabitacion y Habitacion

```
1 @Entity @Table(name = "tipo_habitacion")
2 @Data @NoArgsConstructor @AllArgsConstructor
3 public class TipoHabitacion {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
6     @Column(nullable = false) private String nombre;
7     private String descripcion;
8 }
```

Listing 11: Entidad TipoHabitacion.java

```
1 @Entity @Table(name = "habitacion")
2 @Data @NoArgsConstructor @AllArgsConstructor
3 public class Habitacion {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
6
7     @ManyToOne(fetch = FetchType.LAZY)
8     @JoinColumn(name = "sucursal_id", nullable = false)
9     private Sucursal sucursal;
10 }
```

```
11 @ManyToOne(fetch = FetchType.LAZY)
12 @JoinColumn(name = "tipo_id", nullable = false)
13 private TipoHabitacion tipo;
14
15 @Column(nullable = false) private String numero;
16 @Column(nullable = false) private Integer piso;
17 @Column(nullable = false) private Integer capacidad;
18
19 @Column(name = "precio_noche", nullable = false)
20 private BigDecimal precioNoche;
21
22 @Enumerated(EnumType.STRING)
23 @Column(nullable = false, length = 30)
24 private EstadoHabitacion estado;
25
26 @Column(nullable = false) private Boolean disponible;
27 }
```

Listing 12: Entidad Habitacion.java

13.5. Reserva y tablas intermedias

```
1 @Entity @Table(name = "reserva")
2 @Data @NoArgsConstructor @AllArgsConstructor
3 public class Reserva {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
6
7     @ManyToOne(fetch = FetchType.LAZY)
8     @JoinColumn(name = "cliente_id", nullable = false)
9     private Usuario cliente;
10
11     @Column(name = "fecha_entrada", nullable = false)
12     private LocalDate fechaEntrada;
13
14     @Column(name = "fecha_salida", nullable = false)
15     private LocalDate fechaSalida;
16
17     @Enumerated(EnumType.STRING)
18     @Column(nullable = false, length = 30)
19     private EstadoReserva estado;
20
21     @Column(nullable = false) private BigDecimal total;
22
23     @CreationTimestamp
24     @Column(name = "created_at", nullable = false, updatable = false)
25     private LocalDateTime createdAt;
26 }
```

Listing 13: Entidad Reserva.java

```
1 @Entity @Table(name = "reserva_habitacion")
2 @Data @NoArgsConstructor @AllArgsConstructor
3 public class ReservaHabitacion {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
```

```

6
7     @ManyToOne(fetch = FetchType.LAZY)
8     @JoinColumn(name = "reserva_id", nullable = false)
9     private Reserva reserva;
10
11     @ManyToOne(fetch = FetchType.LAZY)
12     @JoinColumn(name = "habitacion_id", nullable = false)
13     private Habitacion habitacion;
14 }

```

Listing 14: Entidad ReservaHabitacion.java

```

1  @Entity @Table(name = "reserva_servicio")
2  @Data @NoArgsConstructor @AllArgsConstructor
3  public class ReservaServicio {
4      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5      private Long id;
6
7      @ManyToOne(fetch = FetchType.LAZY)
8      @JoinColumn(name = "reserva_id", nullable = false)
9      private Reserva reserva;
10
11      @ManyToOne(fetch = FetchType.LAZY)
12      @JoinColumn(name = "servicio_id", nullable = false)
13      private Servicio servicio;
14
15      @Column(nullable = false) private Integer cantidad;
16
17      @Column(name = "precio_unitario", nullable = false)
18      private BigDecimal precioUnitario;
19
20      @Column(nullable = false) private BigDecimal subtotal;
21 }

```

Listing 15: Entidad ReservaServicio.java

13.6. Pago y Factura

```

1  @Entity @Table(name = "pago")
2  @Data @NoArgsConstructor @AllArgsConstructor
3  public class Pago {
4      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5      private Long id;
6
7      @OneToOne(fetch = FetchType.LAZY)
8      @JoinColumn(name = "reserva_id", nullable = false, unique = true)
9      private Reserva reserva;
10
11      @Column(nullable = false) private BigDecimal monto;
12
13      @Enumerated(EnumType.STRING)
14      @Column(name = "metodo_pago", nullable = false, length = 30)
15      private MetodoPago metodoPago;
16
17      @Enumerated(EnumType.STRING)
18      @Column(nullable = false, length = 30)

```

```
19     private EstadoPago estado;
20
21     @Column(name = "fecha_pago", nullable = false)
22     private LocalDateTime fechaPago;
23 }
```

Listing 16: Entidad Pago.java

```
1 @Entity @Table(name = "factura")
2 @Data @NoArgsConstructor @AllArgsConstructor
3 public class Factura {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
6
7     @OneToOne(fetch = FetchType.LAZY)
8     @JoinColumn(name = "pago_id", nullable = false, unique = true)
9     private Pago pago;
10
11     @Column(name = "numero_factura", nullable = false)
12     private String numeroFactura;
13
14     @Column(nullable = false) private BigDecimal subtotal;
15     @Column(nullable = false) private BigDecimal impuestos;
16     @Column(nullable = false) private BigDecimal total;
17
18     @Column(name = "fecha_emision", nullable = false)
19     private LocalDateTime fechaEmision;
20
21     private String descripcion;
22 }
```

Listing 17: Entidad Factura.java

14. Conclusiones

En esta primera entrega se estableció exitosamente la estructura base del Sistema de Gestión Hotelera:

- Se diseñó un modelo relacional con **12 entidades** normalizadas hasta 3FN.
- Se implementaron **3 migraciones Flyway** que crean, restringen e inicializan la base de datos automáticamente.
- Se configuró la persistencia con **JPA/Hibernate** en modo **validate**, delegando el control del esquema completamente a Flyway.
- La arquitectura de paquetes (entidad, enums) está preparada para recibir en futuras entregas las capas de repositorio, servicio y controlador REST.
- Se incorporaron enumeraciones Java (**EstadoReserva**, **MetodoPago**, **EstadoHabitacion**, etc.) para garantizar dominios controlados tanto en la aplicación como en la base de datos mediante restricciones **CHECK**.